中山大学计算机学院编译器构造实验报告

姓名	学号	实验序号
周子健	23336020	1

实验报告要求：

1. 统一使用此模板格式，导出为PDF提交，命名示例 `张三-23010101-实验1报告.pdf`；
2. 导出为PDF后不得超过8页，超过部分自动截断；
3. 禁止抄袭；
4. 附带导出的会话记录文件，命名示例 `张三-23010101-实验1会话记录.md`；
5. 应体现在实验过程中获得的思考。

一、实验过程

1. 你在实验过程中，有发现什么技巧或心得吗？（例如在LLVM的官方代码中找到了规范或灵感）
2. 你在此次实验中遇到的最难问题和相应解决办法是什么？
    - 在本次实验中，由于一下子接触到的新知识，以及仓库中代码较多，会让人一时不知所措，但是首先搞清楚实验指导中的步骤，再依次阅读对应关键代码，即可理清楚整个实验的脉络。同时，阅读 `clang` 预处理后的代码以及对应 answer，有助于提供实验思路。

补充词法规则

- 对于常见 **关键字**、**标点符号** 和 **运算符**，
    - 首先通过阅读 `answer.txt` 和实验文档，整理所需要识别的字符；
    - 然后通过 DeepSeek 将其格式化输出为 ANTLR `.g4` 要求的词法规则的基本格式；
    - 再使用 DeepSeek 格式化输出为 C++ 代码中映射代码，将词法规则名正确映射为 `clang` 定义的 `Token` 类型描述符。

  会话记录文件见 `关键字转换.md`。
- **数值常量**：包括十进制、八进制、十六进制整数和十进制、十六进制浮点数。其中需要补充十六进制整数和十进制、十六进制浮点数。
    - **十六进制整数**：使用正则表达式描述为 `0[xX][0-9a-fA-F]+`，编写 ANTLR 规则为

      ```
      fragment
      HexadecimalConstant
          : '0'[xX] [0-9a-fA-F]+
          ;
      ```

    - **十进制浮点数**：使用正则表达式描述为

      ```
      [0-9]*\.[0-9]+ ([eE][+-]?[0-9]+)? [fFlL]?
      [0-9]+\.[0-9]* ([eE][+-]?[0-9]+)? [fFlL]?
      [0-9]+ [eE][+-]?[0-9]+ [fFlL]?
      ```

编写 ANTLR 规则为

```
fragment
DecimalFloatConstant
    : [0-9]* '.' [0-9]+ ([eE][+-]?[0-9]+)?
    | [0-9]+ '.' [0-9]* ([eE][+-]?[0-9]+)?
    | [0-9]+ [eE][+-]?[0-9]+
    ;
```

- **十六进制浮点数**：使用正则表达式描述为

```
0[xX][0-9a-fA-F]*\.[0-9a-fA-F]+ [pP][+-]?[0-9]+ [fFlL]?
               0[xX][0-9a-fA-F]+\.[0-9a-fA-F]* [pP][+-]?[0-9]+ [fFlL]?
               0[xX][0-9a-fA-F]+ [pP][+-]?[0-9]+ [fFlL]?
```

编写 ANTLR 规则为

```
fragment
HexadecimalFloatConstant
    : '0' [xX][0-9a-fA-F]* '.' [0-9a-fA-F]+ [pP][+-]?[0-9]+ [fFlL]?
    | '0' [xX][0-9a-fA-F]+ '.' [0-9a-fA-F]* [pP][+-]?[0-9]+ [fFlL]?
    | '0' [xX][0-9a-fA-F]+ [pP][+-]?[0-9]+ [fFlL]?
    ;
```

- 对于 `SYsULexer.g4` 中原本包含的十进制、八进制和十六进制整数，原本并不包括可选后缀，因此进行补充：

```
[1-9][0-9]*([uU]?[lL]{0,2}|[lL]{0,2}[uU]?)              // 十进制整数
0[0-7]*([uU]?[lL]{0,2}|[lL]{0,2}[uU]?)                      // 八进制整数
0[xX][0-9a-fA-F]+([uU]?[lL]{0,2}|[lL]{0,2}[uU]?)    // 十六进制整数
```

由于后缀都是相同的，所以只需要在 ANTLR `.g4` 文件中对应的条目后添加 `([uU]? [lL]? [lL]? | [lL]? [lL]? [uU]?)` 后缀即可。

更新状态信息

- 识别 **换行**：
  - 首先取消 `SYsULexer.g4` 中识别换行规则的 `-> skip`，这样识别到换行后会生成 token，词法规则名为 `Newline`；
  - 定义 `static bool startOfLine` 变量，使得其可以在 `print_token()` 函数的多次调用中传递；
  - 当输出非换行符和空格 token 时，若 `startOfLine==true`，则输出 `[StartOfLine]` 信息，并设置 `startOfLine=false`；同时需要将代码行计数器 `lineNumber` 加一。

  通过以上逻辑实现 `[StartOfLine]` 信息的输出。
- 识别 **空格**：
  - 首先取消 `SYsULexer.g4` 中识别空格规则的 `-> skip`，这样识别到空格后会生成 token，词法规则名为 `Whitespace`；
  - 定义 `static bool leadingSpace` 变量，使得其可以在 `print_token()` 函数的多次调用中传递；

- 当输出非换行符和空格 token 时，若 `leadingSpace==true`，则输出 `[LeadingSpace]` 信息，并设置 `leadingSpace=false`。

通过以上逻辑实现了 `[LeadingSpace]` 信息的输出。

同时，利用 `leadingSpace` 来统计行号信息 `static size_t lineNum`，当 `leadingSpace==true` 时将 `lineNum++` 以统计当前行行号。

- 处理 **行标记**：我们需要从行标记中获取当前代码文件的 **文件路径** 和 **起始行号**。

  行标记的基本格式是 `# linenum filename flags`，它的含义是**下一行**来自于 `filename` 文件的第 `linenum` 行。通过观察，可以发现最后一行行标记即为当前代码的文件路径和起始行，同时 `flags` 为 2。

  故每次解析行标记信息，并保留最后一条行标记信息：

```
static LineMarker lastLineMark;
static size_t lineNumber = 0;
if (tokenTypeName == "LineAfterPreprocessing") {
    lastLineMark = parseLineMark(token->getText());
    lineNumber = lastLineMark.lineNumber - 1;
    return;
}
std::string locInfo = " Loc=<" + lastLineMark.filename + ":" +
                          std::to_string(lineNumber) + ":" +
                          std::to_string(token->getCharPositionInLine() + 1) +
                          ">";
```

  其中 `parseLineMark()` 函数解析行标记并返回 `LineMarker` 结构体类型，定义如下：

```
struct LineMarker {
    int lineNumber;
    std::string filename;
    int flags[4];
};

LineMarker parseLineMark(const std::string& line) {
}
```

  对于 `parseLineMark` 函数，使用 vibe Coding 补全，使用非正则表达式匹配方式对行标记进行分解。

评测

除基准评测之外，还定义了 `diy-case1.sysu.c` 用于测试基准测例中未包含的情况，具体见代码：

```
int sum(int count, ...);    // 测试 ...
int main(){
    // This is a text
    /* This is a test */
    int x = 0;
    x <<= 1 << 2 < 3;   // 测试 <<=
    x >>= 2 >> 1 > 1;   // 测试 >>=
    x + x++;    // 测试 ++, --
    x - x--;
    x + ++x;
```

```
    x - --x;
    int inty = 1 << 2 + (x <<= 2) <= 4; // 测试 <=
    float z = 0.1;  // 测试浮点数
    float a0x = 0.0 <= 1e-10 + 12.0e+1 - 10.00e2 * 1e+2;    // 测试科学计数法

    unsigned long long ull_x = 1llu;    // 测试整型后缀
    unsigned long ul_y = 2ul;

    return 0;
}
```

最后均通过了全部基准测试和自定义测试：

```
[build] ================================================================================
[build] diy-case1.sysu.c ............................................ 100.00/100.00
[build] diy-case2.sysu.c ............................................ 100.00/100.00
[build] diy-case3.sysu.c ............................................ 100.00/100.00
[build] diy-case4.sysu.c ............................................ 100.00/100.00
[build] diy-case5.sysu.c ............................................ 100.00/100.00
[build]
[build] task1
[build] 总分（加权）：100.00/100.00
[build] ================================================================================
```

二、AI使用

1. 你使用了什么模型，使用了多少token?

   本次实验中主要使用 deepseek-chat 模型。其中，补全 `parseLineMark()` 函数的单词 token 用量为 27.8k。

2. 你在实验过程中的AI使用方面有什么心得吗（prompt、coding、chat）？（例如你认为效果最好的prompt）

   补全代码时，最好直接说明需要的实现方式（如使用/不使用正则表达式，使用什么库）；同时，给是适当的样例有助于大模型正确理解。例如：

   ```
   请帮我补全代码，完成对行标记的处理。注意行标记的默认格式为 # linenum filename flags，其中 flags 可能有零个、一个或多个数字 (1,2,3,4)，用空格分隔。以下为行标记的例子：
   # 1 "./functional-0/000_main.sysu.c"
   # 1 "<built-in>" 1
   # 1 "<built-in>" 3
   # 384 "<built-in>" 3
   # 1 "<command line>" 1
   # 1 "<built-in>" 2
   # 1 "./functional-0/000_main.sysu.c" 2
   请帮我补全parseLineMark函数：只需要填充该函数，不要修改其他代码；不使用正则表达式库；在你添加代码中，添加适当的中文注释
   ```

   如果尝试直接将整个实验的prompt丢给大模型，可能导致大模型消耗大量token且难以推进任务，将实验任务彻底理解后再分为子任务交给大模型完成，成功率更高。

三、实验建议

1. 你对实验内容有什么建议吗（配套资料、实验内容、自动脚本）？（例如在自动脚本方面有其它需求）

   希望能够使用本地 vscode 远程连接容器，使用网页端开发导致快捷键使用有一定障碍。

2. 你对实验提供的AI服务的方面有什么建议吗（Agent、模型、功能）？